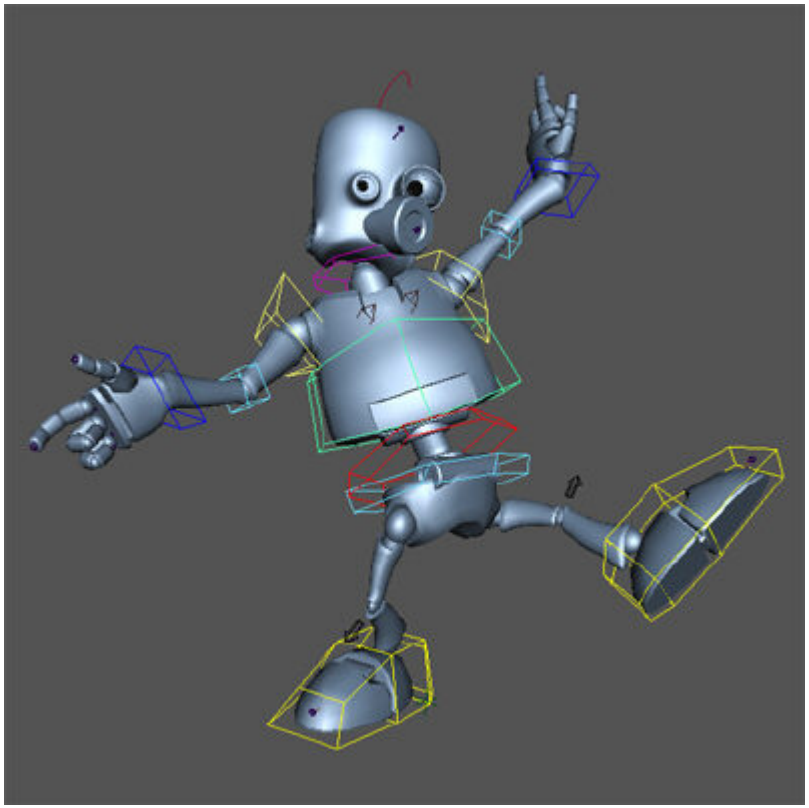
GOOD LUCK IN YOUR FINAL



- Quiz!!!!!!

- Write a complete program that does these actions:
- • asks the user for the the name of a data file and opens that file.
- • asks the user for how many numbers to expect in the file (integers).
- • allocates a dynamic array of exactly the size required.
- • displays the numbers present in the even cells [0],[2],[4],….

```
#include <stdio.h>
#include <stdlib.h>
int  main (void) {
char filename;
int n, *array, i;
FILE *in;
printf ("File name: ");
scanf {"%s", filename);
in = fopen (filename, "r");
printf ("How many numbers? ");
scanf ("%d", &n);
array = (int *) calloc (n, sizeof (int));
for (i=0; i<n, ++i)
{
fscanf (in, "%d", &array[i]);
if (i%2==0)
printf ("%d ", array[i]);
}
fclose (in);
free (array);
return (0);
}
```

Complete the function only (no main program) that will take two matrices (up to 5x5 in size) and give their multiplication.
Assume that the matrices coming from the main program are already compatible for multiplication.

Complete the function only (no main program) that will take two matrices (up to 5x5 in size) and give their multiplication.
Assume that the matrices coming from the main program are already compatible for multiplication.

```
void
matmult (int m1[][5], int m2[][5], int m3[][5])
{
int i,j,k;
for (i=0; i<5; ++i)
for (j=0; j<5; ++j)
{
m3[i][j]=0.0;
for (k=0; k<5; ++k)
m3[i][j] = m3[i][j] + m1[i][k] * m2[k][j];
}
}
```

Write a function only that takes a 2D array of integers and returns another 2D array of integers with its corner values swapped (the top-left with bottom-right and the top-right with bottom-left). In addition, the function will return the sum of all four corner values. Do not scan nor print anything in the function. Everything comes from and goes back to the main program. The maximum possible array size is 50x50 but in can be as small as 2x2. See the example with a 3x3 array.

Ex: if the array is 1 2 3 then the new array will be 9 2 7

4 5 6	4 5 6
7 8 9	3 8 1

and the sum of the corners will be 20.

Write a function only that takes a 2D array of integers and returns another 2D array of integers with its corner values swapped (the top-left with bottom-right and the top-right with bottom-left). In addition, the function will return the sum of all four corner values. Do not scan nor print anything in the function. Everything comes from and goes back to the main program. The maximum possible array size is 50x50 but in can be as small as 2x2. See the example with a 3x3 array.

Ex: if the array is 1 2 3 then the new array will be 9 2 7

4 5 6 4 5 6

7 8 9 3 8 1

and the sum of the corners will be 20.

```
int
swapcorners (int ar1[][50], int ar2[][50], int nrows, int
ncols)
{
int sumcorners, temp;
ar2[nros-1][ncols-1] = ar1[0][0];
ar2[0][0] = ar1[nros-1][ncols-1];
ar2[nrows-1][0] = ar1[0][ncols-1];
ar2[0][ncols-1] = ar1[nrows-1][0];
sumcorners = ar1[0] + ar1[nros-1][ncols-1] + ar1[0]
[ncols-1] + ar1[nrows-1][0];
return (sumcorners);
}
```

```
#include <stdio.h>
double cps125 (double p, double *q, int r )
{
double s;
s = p - 10;
*q = s - p *3 /r;
return (s+1);
}
int
main (void)
{
int m=2;
double  mm=20, nn  ;
mm= cps125 (mm, &nn, m);
printf ("%lf", mm+nn);
return (0);
}
```

```
#include <stdio.h>
double wilma (double p, double q)
{
p = p-q;
return (p-q);
}
double fred (double fredi)
{ double m,n;
fredi = wilma( m, n) -300;
return (fredi/2);
}
int
main (void)
{
double mm=19,nn=20;
nn = wilma (nn*2,mm)- wilma (mm, nn+1);
printf ("%lf", nn);
return (0);
}
```

OUTPUT?

```
#include <stdio.h>
typedef struct{
int a, b;
}abc;
abc
func1 (abc fred, int wilma, int *dino){
fred.a = wilma + 1;
while (wilma > 0){
fred.b = *dino + wilma+1;
wilma = wilma - 1;
*dino=*dino+1;
}
return (fred);}
int main (void){
abc x;
int y=2, z=3;
x = func1 (x, y, &z);
printf ("%d %d\n", x.a, x.b);
printf ("%d %d\n", y, z);
return(0);
}
```

What is the output of the following program?

```
#include <stdio.h>
void wilma (double* ptr)
{
if (*ptr < 100.0 && * ptr >0.0)
* ptr = * ptr /2;
}
int
main (void)
{
double aa=2.0;
aa=aa*3;
wilma(&aa);
printf ("%lf", aa);
return (0);
}
```

Output?

```
#include <stdio.h>
int wilma (int aa)
{
int i;
aa= aa +1;
if (aa <12)
aa = wilma (aa)+1;
return (aa);
}
int
main (void)
{
int p=10;
p = wilma (p);
printf ("%d", p);
return(0);}
```

Output?

```
#include <stdio.h>
int cps (int mark, int *num)
{
int gp;
gp = mark - 5;
*num = gp + mark;
return (gp);
}
int
main (void)
{
int tt=10, vv;
tt = cps (tt, &vv);
printf ("%d", tt+vv);
return (0);
}
```

Output?

```
#include <stdio.h>
int cps (int mark)
{
mark = mark-3;
return (mark);
}
int ckcs (int num)
{
num= cps(num) /2;
return (num);
}
int
main (void)
{
int t=12;
t = ckcs (t)-1;
printf ("%d", t);
return (0);
}
```

What is the output of the following program (one answer per line)?

```
#include <stdio.h>
#include <string.h>
typedef struct beaches {
int number ;
   struct beaches* next;
}beach;
int main (void){
        beach b1, b2, b3, b4, b5, b6;
        beach *ptr;
        b1.number = 27;
        b2.number = 45;
        b3.number = 61;
        b4.number = 36;
        b5.number = 80;
        b6.number = 90;
        b6.next=&b5
         ptr = &b2;
        b1.next = NULL;
        b3.next = &b1;
        b4.next = &b3;
        b2.next = &b4;
        b5.next = &b6;
       while (ptr != NULL){
              printf ("%d\n", ptr -> number);
              ptr = ptr -> next;       }                return (0);}
```

```
#include <stdio.h>
#include <string.h>
typedef struct f {
int b[2];
   struct f* next;
}f;
int main (void){   f   f1, f2, f3, f4;
        f  *ptr;
        f4.next = &f2;  f1.b[0]=3;   f1.b[1]=5;   f2.b[0]=7;  f2.b[1]=6;
                                    f2.next = &f1;
                                    f3.b[0]=2;
          f3.b[1]=4;
                                   f4.b[0]=9;
                                    f4.b[1]=8;
        f1.next = NULL;
        ptr=&f4;

       while (ptr != NULL){
              printf ("%d\n", ptr -> b[1]);
              ptr = ptr -> next;       }                return (0);}
```

```
#include <stdio.h>
#include <string.h>
typedef struct beach1 {
char name[20];
int capacity;
}beach1;
typedef struct beach2 {
char name[20];
double temrature;
int capacity;
}beach2;
int
main (void)
{
beach1 t = {"Florida", 10000};
beach2 y= {"Miami",35.0,20000};
beach1* ptr1;
beach2* ptr2;
char name[20];
double temp;
ptr1 = &t;
ptr2 = &y;
temp=y.temrature;
printf ("temrature %lf", temp);
return (0);
}
```

/* The following program demonstrates usage of structure */

```
#include <stdio.h>
typedef struct
{
int p, q, rotating ;
}moon;
moon google (moon cps125, int x, int *z){
cps125.p= x + 35;
while (x >= 42)
{
cps125.q = *z + x;
x = x - 1;
}
return (cps125); }
int main (void) {
moon venus;
int tt=43, dd=58 ;
venus = google (venus, tt, &dd);
while(tt<100){
printf ("%d %d\n", venus.p,venus.q);
return(0);
}}
```

Declaring arrays

- You can also put the values in the array while declaring it:
  - double a[5]={1.3, 2.4, 6.2, 4.5, 1.1};

- When declaring and initializing, the size is then optional:
  - double b[ ]={6.78, 7.88, 4.55, 1.33};

  - What is the size of array **b**?

Cell numbers

- A cell is always referenced by its cell number (also called subscript or offset).
- **age[2]** is actually the <u>third</u> cell of the array called age.
- Note that int **age[6],** Since age is size 6, then its cells are numbered from 0 to 5 only.

Arrays and cells

- **age** being an array, a statement like **age=70;** would be illegal as age would be an address, not a value. With a value, you must always mention which cell to fill, so **age[3]=70;** would be correct.

- The sizeof operator will return the size (in bytes) of the array.

- **printf ("%d", sizeof (age));** would display 24 on a 32-bit system (6 cells of integers at 32 bits or 4 bytes each).

Sending arrays to functions

```
find (int array[], int size)
{
int i, largest, where;
largest = array [0];
for (i=0; i<size; ++i)
   if (array[i]>largest)
   {
          largest = array[i];
          where = i;
      }
return (where);
}
```

*** There is something missing in this program, can you find out what?

Sending arrays to functions

```
/* the main program */
int
main (void)
{
    int a[] = {6,2,8,7,3};
    int location;
    location = find (a, 5);
    printf (“Cell #%d\n”, location);
    return (0);
}
```

Having an array as a “result”

```
/* the main program */
int
main (void)
{
    int x[] = {1,2,3,4}, i;
  int y[] = {10,20,30,40};
  int z[4], i;
    addarrays (x, y, z, 4);
    for (i=0; i<4; ++i)
       printf ("%4d", z[i]);
    return (0);
}
```

Sending arrays to functions

(multiple “results”)

```
/* function provides the smallest value in an
array and its location */

find_small (int array[], int size, int *smallest)
{
 int i, where;
 *smallest = array [0];
   where = 0;
for (i=0; i<size; ++i)
   if (array[i] < *smallest)
   {
           *smallest = array[i];
           where = i;
      }
return (where);
}
```

Sending arrays to functions

(multiple “results”)

```
/* the main program */
int
main (void)
{
    int a[] = {6,2,8,7,3};
    int location, small;
    location = find_small (a, 5, &small);
    printf (“Smallest number: %d\n”,
small);
    printf (“Location: %d\n”, location);
    return (0);
}
```

Dynamic allocation of arrays

- Arrays, as we know them now, can only be of a fixed size. **int x[100];** declares an array size 100, no less no more.

```
printf ("Enter the size of the array: ");
scanf ("%d", &size);
int x[size];
```

Dynamic Allocation of Arrays

- It is impossible to have arrays of varying sizes because that makes programming inefficient. C is very strict with arrays but it is also the most efficient of all the high-level languages.

- There is a way to have dynamic allocation in C. The solution is to use "heap" memory instead of the usual memory we have been using so far.

- Heap memory: slower, less-structured, dynamic memory.

Using a dynamic array

```
int size;
/* 1. declare a pointer to the array */
double *array;

printf ("Enter the size of the array: ");
scanf ("%d", &size);

/* 2. allocate the array in the heap */
array = (double *) calloc (size,
  sizeof(double));

/* 3. use the array normally */
...
/* 4. free the heap memory */
free (array);
```

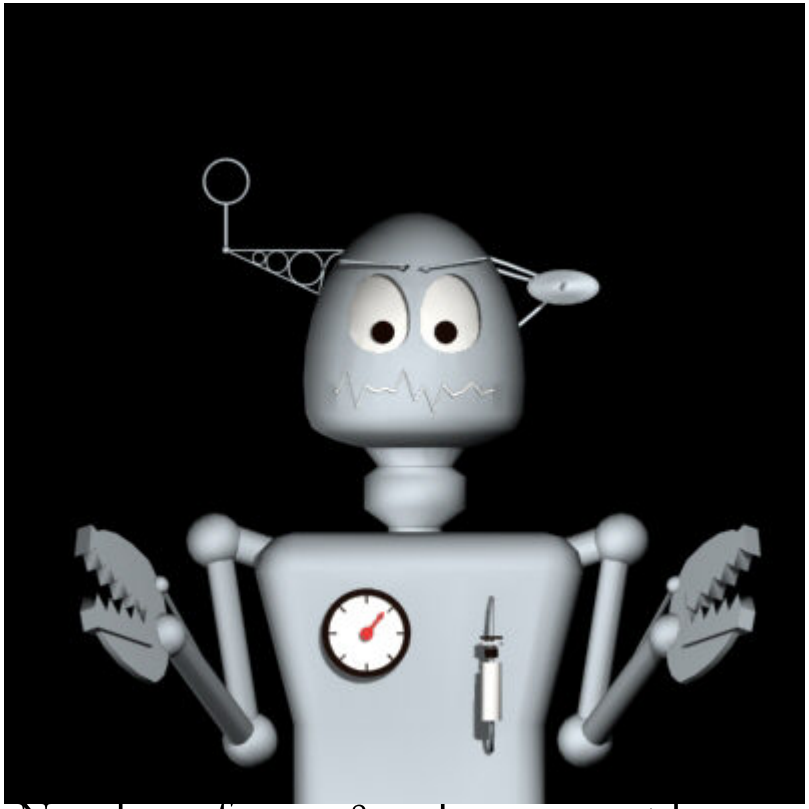
- Write a complete program that fills an array 5x6 of integers from a file and prints out the numbers in the main diagonal.

```
#include <stdio.h>
int
main (void) {
int a[5][6], i, j;
FILE *in;
in = fopen ("file.txt", "r");
for (i=0; i<5; ++i)
for (j=0; j<6; ++j)
fscanf (in, "%d", &a[i][j]);
fclose (in);
for (i=0; i<5; ++i)
printf ("%d ", a[i][i]);
return (0);
}
```

- Program a function that takes a string of lowercase letters and return the same string with lowercase "a" changed into an uppercase "A" and any combination of "to" changed into "TO" (do not change t and o alone). The function prototype is
  void q1 (char string[ ]);

```
void q1 (char string[]){
        int i=0;
        while (string[i]!='\0')
        {
                if (string[i]=='a')
                        string[i] = 'A';
                if (string[i]=='t' && string[i+1]=='o')
                {
                        string[i] = 'T';
                        string[i+1] = 'O';
                }
                i=i+1;
        }
}
```

How to transpose a matrix using only ONE matrix?



- Requirements: Number of rows & columns must be equal, i.e., a square matrix

/* TRASPOSE A AND PUT IT BACK INO A */

```
/* PROGRAM # 88 */
/*TRASPOSE A AND PUT IT BACK INO A */
#include<stdio.h>
#define   asize          3
/* DECLARE FUNCTION PROTOTYPES */
void GetMatrix(int A[ ][asize]);
void MatrixTranspose1(int A[ ][asize]);
void PrintMatrix(int A[ ][asize]);
main( ){
int MatrixA[asize][asize];
/* Input the matrix from the user, display it */
GetMatrix(MatrixA);
PrintMatrix(MatrixA);
/*Transpose matrix A, put it back to A, display A */
MatrixTranspose1(MatrixA);
PrintMatrix(MatrixA);}
/*TO READ A MATRIX FROM THE USER */
void GetMatrix(int A[ ][asize]){
int i, j;
for(i= 0; i<asize; i++)
                    for(j= 0; j<asize; j++){
                              printf("A[%d][%d]: ”, i, j);
                              scanf(“%d”, &A[i][j]);          }
}
```

/* TRASPOSE A AND PUT IT BACK INO A */

```
/* TO TRANSPOSE A SQUARE MATRIX */
void MatrixTranspose1(int A[ ][asize]){
int i, j, temp;
for(i=0; i<asize; i++)
for(j=i; j<asize; j++){
temp=A[j][i];
A[j][i]=A[i][j];
A[i][j]=temp;
}
}
/* TO PRINT A MATRIX IN THE FORM OF A MATRIX */
void PrintMatrix(int A[ ][asize]){
int i, j;
puts("");
        for(i= 0; i<asize; i++){
                for(j= 0; j<asize; j++)
                        printf("%5d", A[i][j]);
                printf("\n");
        }}
```

Write a complete program reading from a file numbers (the first number being the quantity of the other numbers to read). Place these last numbers in an array. All numbers are integers. Write a function taking this array as input and returning the largest number as output. The function prototype will be

```
int largest (int array[ ], int size);
```

Write a complete program reading from a file numbers (the first number being the quantity of the other numbers to read). Place these last numbers in an array. All numbers are integers. Write a function taking this array as input and returning the largest number as output. The function prototype will be

int largest (int array[ ], int size);

```
#include <stdio.h>
int largest (int array[], int size){
        int i;
        int large;
        large = array[0];
        for (i=0; i<size; +i)
                if (array[i]>large)
                        large=array[i];
        return (large);}
int main (void){
        int n, array[100], i, big;
        FILE *in;
        in = fopen ("file.txt", "r");
        fscanf (in, "%d", &n;);
        for (i=0; i<n; +i)
                fscanf (in, "%d", &array[i]);
        big = largest (array, n);
        printf ("The largest number is %d", big);
        fclose (in);
        return (0);}
```

We have a program that has three parallel arrays: barcode (an array of integers), description (an array of strings) and price (an array of doubles). The maximum size of the arrays is 100. The program will display the price and description on when you enter a specific barcode. Write a function only that will send back a price and a description to the main when the barcode is sent to it. The function prototype will be

double scanner (char descr[100][30], double price[100], int barcode[100], int scancode, int* location);

We have a program that has three parallel arrays: barcode (an array of integers), description (an array of strings) and price (an array of doubles). The maximum size of the arrays is 100. The program will display the price and description on when you enter a specific barcode. Write a function only that will send back a price and a description to the main when the barcode is sent to it. The function prototype will be

double scanner (char descr[100][30], double price[100], int barcode[100], int scancode, int* location);

```
double
scanner (char descr[100][30], double price[100], int barcode[100], int scancode, int* location)
{
    int i;
    double howmuch;
    for (i=0; i<100; ++i)
        if (barcode[i] == scancode)
        {
            *location = i;
            howmuch = price[i];
        }
    return (howmuch);
}
```

Write a complete program that fills an array 5x6 of integers from a file and prints out the numbers in the main diagonal.

Write a complete program that fills an array 5x6 of integers from a file and prints out the numbers in the main diagonal.

```
#include <stdio.h>
int
main (void)
{
    int a[5][6], i, j;
    FILE *in;
    in = fopen ("file.txt", "r");
    for (i=0; i<5; ++i)
        for (j=0; j<6; ++j)
            fscanf (in, "%d", &a[i][j]);
    fclose (in);
    for (i=0; i<5; ++i)
        printf ("%d   ", a[i][i]);
    return (0);
}
```

A text file called accidents.bin contains accident numbers on the main highways of Ontario for a typical year. Write a complete program (main only) that will read data from the file (we don't know how many highways in the file but there is only one year) and calculate the total number of accidents in the year. Print out the year and the total number of accidents. You must use the userdefined structure type accdata in your program to store the data.

Ex: if the file contains 2004

401	400
402	70
403	80

your report will print 550 accidents in 2004.

A text file called accidents.bin contains accident numbers on the main highways of Ontario for a typical year. Write a complete program (main only) that will read data from the file (we don't know how many highways in the file but there is only one year) and calculate the total number of accidents in the year. Print out the year and the total number of accidents. You must use the userdefined structure type accdata in your program to store the data.

Ex: if the file contains 2004

401	400
402	70
403	80

your report will print 550 accidents in 2004.

```
#include <stdio.h>
typedef struct{
int highwayno; /* highway number */
int nacc; /* number of accidents */
} accdata;
int main (void){
FILE *in;
int totacc=0, year;
accdata data;
in = fopen ("accidents.bin", "r");
fscanf (in, "%d", &year);
while (fscanf (in, "%d%d", &data.highwayno, &data.nacc)!=EOF){
totacc = totacc + data.nacc;}
printf (%d accidents in %d\n", totacc, year);
fclose (in);
return (0);
}
```

Write a complete C program consisting only of a main program that will read from a file (named *pairs.txt) pairs of numbers:* a double followed by an integer. We do not know how many pairs there are but all pairs are complete. Your program must calculate pair by pair the division of the double by the integer and print on a separate line each quotient with three decimals precision. Finally your program will display on a separate line the number of pairs followed by the word "pairs.". For example if the file contains **4.5 3 6.6 2, your** program will display

1.500
3.300
2 pairs.

Write a complete C program consisting only of a main program that will read from a file (named *pairs.txt) pairs of numbers:* a double followed by an integer. We do not know how many pairs there are but all pairs are complete. Your program must calculate pair by pair the division of the double by the integer and print on a separate line each quotient with three decimals precision. Finally your program will display on a separate line the number of pairs followed by the word "pairs.". For example if the file contains **4.5 3 6.6 2, your** program will display
1.500
3.300
2 pairs.

```
#include <stdio.h>
int main (void){
int npairs=0, integer;
double doubl;
FILE *in;
in = fopen ("pairs.txt", "r");
while (fscanf(in, "%lf %d", &doubl, &integer)!=EOF)
{
npairs = npairs + 1;
printf ("%.3lf\n", doubl/integer);
}
fclose (in);
return (0);
}
```

Complete the function only (no main program) that will take in an array of double numbers and will "return" a different array containing the inverted values (1/n). For example if the original array contains 1.0, 2.0, 3.0..., the inverted array will contain 1.0, 0.5, 0.3333... Your function must work with arrays of any size (same size both arrays). Your function must also return the value of the last cell of the inverted array.

Complete the function only (no main program) that will take in an array of double numbers and will "return" a different array containing the inverted values (1/n). For example if the original array contains 1.0, 2.0, 3.0..., the inverted array will contain 1.0, 0.5, 0.3333... Your function must work with arrays of any size (same size both arrays). Your function must also return the value of the last cell of the inverted array.

```
double
inverting ( double a1[], double a2[], int size )
{
int i;
for (i=0; i<size; ++i)
a2[i] = 1 / a1[i];
return ( a2[i] );
}
```

Review more!

- Write a complete program that will read numbers from a file (we do not know how many numbers).
- Have the program add up all the negative numbers only.
-

```
#include <stdio.h>
int
main (void)
{       int n, status, sum = 0;
        status = scanf ("%d", &n);
        while (status != EOF)
        {
                if (n < 0)
                        sum = sum + n;
                status = scanf ("%d", &n);
        }
        printf (Sum is "%d\n", sum);
        return (0);
}
```

- Complete the function only (no main program) that takes two square matrices of integers (up to 50x50 in size) and calculates the scalar product of the two main diagonals.
  For example, for the two following matrices,
- 1 2 3 9 1 2
- 4 5 6 3 4 5
- 7 8 9 6 7 8
- the scalar product would be 1x9 + 5x4 + 9x8 = 101. Note that the scalar product will be returned to the main, not printed out in the function.
- int
- scaldiag ( )
- {

- return ( );
- }
- Indicate the call to the function in the main program assuming the matrices are called mat1 and mat2 and the scalar product is called sp.

- _____

- Write a complete program that will read the following data from a file called **tv.dat. The format is the following: the first data is a integer indicating how many words will follow. Here is a typical file:**
- 3 my three sons
- 2 star trek
- 1 friends
- 2 home improvement
- 1 lost
- 4 hockey night in canada
- 2 the national
- Your program must display all the tv show titles that have 3 words or more.

```
#include <stdio.h>
int  main (void) {
int nw, i;
FILE *in;
in = fopen ("tv.dat", "r");
while (fscanf (in, "%d", &nw)!= EOF) {
for (i=0; i<nw; ++i) {
fscanf (in, "%s", word);
if (nw>=3)
printf ("%s", word); } }
fclose (in);
return (0); }
```

- Write a complete program (main only) that will read the following book titles from a file called **book.dat.** The format of the file is : The first data is a integer indicating how many words will follow(no words are longer than 15 letters). We do not know how many lines are in the file. Here is a typical file:
- 5 the left hand of darkness
- 3 the lost world
- 5 two years before the mast
- 4 peter oan and wendy
- 2 moby dick
- Your program must display only the words that have exactly five (5) letters, displaying each word on a separate line. In the preceding file example, your program would print out
- world
- years
- peter
- wendy
- Do not use arrays of strings(2D arrays).
- You may use either redirection (scanf)or FILE I/O (fscanf).

Write a complete program reading from a file numbers (the first number being the quantity of the other numbers to read). Place these last numbers in an array. All numbers are integers. Write a function taking this array as input and returning the largest number as output. The function prototype will be

int largest (int array[ ], int size);

```
#include <stdio.h>
int largest (int array[], int size){
        int i;
        int large;
        large = array[0];
        for (i=0; i<size; +i)
                if (array[i]>large)
                        large=array[i];
        return (large);}
int main (void){
        int n, array[100], i, big;
        FILE *in;
        in = fopen ("file.txt", "r");
        fscanf (in, "%d", &n;);
        for (i=0; i<n; +i)
                fscanf (in, "%d", &array[i]);
        big = largest (array, n);
        printf ("The largest number is %d", big);
        fclose (in);
        return (0);}
```

We have a program that has three parallel arrays: barcode (an array of integers), description (an array of strings) and price (an array of doubles). The maximum size of the arrays is 100. The program will display the price and description on when you enter a specific barcode. Write a function only that will send back a price and a description to the main when the barcode is sent to it. The function prototype will be

double scanner (char descr[100][30], double price[100], int barcode[100], int scancode, int* location);

```
double
scanner (char descr[100][30], double price[100], int barcode[100], int scancode, int* location)
{
    int i;
    double howmuch;
    for (i=0; i<100; ++i)
        if (barcode[i] == scancode)
        {
            *location = i;
            howmuch = price[i];
        }
    return (howmuch);
}
```

Write a complete program that fills an array 5x6 of integers from a file and prints out the numbers in the main diagonal.

```
#include <stdio.h>
int
main (void)
{
    int a[5][6], i, j;
    FILE *in;
    in = fopen ("file.txt", "r");
    for (i=0; i<5; ++i)
        for (j=0; j<6; ++j)
            fscanf (in, "%d", &a[i][j]);
    fclose (in);
    for (i=0; i<5; ++i)
        printf ("%d   ", a[i][i]);
    return (0);
}
```

A text file called accidents.bin contains accident numbers on the main highways of Ontario for a typical year. Write a complete program (main only) that will read data from the file (we don't know how many highways in the file but there is only one year) and calculate the total number of accidents in the year. Print out the year and the total number of accidents. You must use the userdefined structure type accdata in your program to store the data.

Ex: if the file contains 2004

401	400
402	70
403	80

your report will print 550 accidents in 2004.

```
#include <stdio.h>
typedef struct{
int highwayno; /* highway number */
int nacc; /* number of accidents */
} accdata;
int main (void){
FILE *in;
int totacc=0, year;
accdata data;
in = fopen ("accidents.bin", "r");
fscanf (in, "%d", &year);
while (fscanf (in, "%d%d", &data.highwayno, &data.nacc)!=EOF){
totacc = totacc + data.nacc;}
printf (%d accidents in %d\n", totacc, year);
fclose (in);
return (0);
}
```

Write a complete C program consisting only of a main program that will read from a file (named *pairs.txt) pairs of numbers:* a double followed by an integer. We do not know how many pairs there are but all pairs are complete. Your program must calculate pair by pair the division of the double by the integer and print on a separate line each quotient with three decimals precision. Finally your program will display on a separate line the number of pairs followed by the word "pairs.". For example if the file contains **4.5 3 6.6 2, your** program will display
1.500
3.300
2 pairs.

```
#include <stdio.h>
int main (void){
int npairs=0, integer;
double doubl;
FILE *in;
in = fopen ("pairs.txt", "r");
while (fscanf(in, "%lf %d", &doubl, &integer)!=EOF)
{
npairs = npairs + 1;
printf ("%.3lf\n", doubl/integer);
}
fclose (in);
return (0);
}
```

Complete the function only (no main program) that will take two matrices (up to 5x5 in size) and give their multiplication.
Assume that the matrices coming from the main program are already compatible for multiplication.

```
void
matmult (int m1[][5], int m2[][5], int m3[][5])
{
int i,j,k;
for (i=0; i<5; ++i)
for (j=0; j<5; ++j)
{
m3[i][j]=0.0;
for (k=0; k<5; ++k)
m3[i][j] = m3[i][j] + m1[i][k] * m2[k][j];
}
}
```

Complete the function only (no main program) that will take in an array of double numbers and will "return" a different array containing the inverted values (1/n). For example if the original array contains 1.0, 2.0, 3.0..., the inverted array will contain 1.0, 0.5, 0.3333... Your function must work with arrays of any size (same size both arrays). Your function must also return the value of the last cell of the inverted array.

```
double
inverting ( double a1[], double a2[], int size )
{
int i;
for (i=0; i<size; ++i)
a2[i] = 1 / a1[i];
return ( a2[i] );
}
```

- A simplified version of the matrix multiplication for 2x2 matrices can be represented as:

$$\overset{\text{mat1}}{\begin{bmatrix} a & b \\ c & d \end{bmatrix}} \times \overset{\text{mat2}}{\begin{bmatrix} e & f \\ g & h \end{bmatrix}} \rightarrow \overset{\text{mat3}}{\begin{bmatrix} ae+bg & af+bh \\ ce+dg & cf+dh \end{bmatrix}}$$

- The answer to this question should be a C program structured as a helper (user-defined) function only (do not write the preprocessor directives not the main program). Your function named *mult performs* the 2x2 matrix multiplication and displays on the screen the resulting matrix in conventional matrix format (2 rows / 2 columns). All variables are integers. You must use 2-D arrays only for this question and name them mat1, mat2 and mat3. No other variables are permitted (except i and j as loop control variables if needed). Do not use dynamically allocated variables. Your function needs to work only with 2x2 matrices. *Penalties will be applied for bad variable naming.*
- void
- mult ( )
- {
- }

- Write a complete C program (main only) that asks the user for the size of an array of integers, allocates dynamically a one-dimension array of that size and then fills the array by asking the numbers at the keyboard (use proper prompt for each entry).
- Your program will then traverse the array to check if two adjacent cells contain the same number. If you find some, print out the cell numbers and the value.
- For example if the array contains the values
- 34,66,77,99,65,65,24,24,24,22 the program will print
- Cells 4 and 5 contain 65.
- Cells 6 and 7 contain 24.
- Cells 7 and 8 contain 24.

Review

& Extra Practice

HOMEWORK!!!

- **How to Add or Subtract Two Vectors?**
- /* ADD OR SUBTRACT TWO ARRAYS */
- /* TWO ARRAYS MUST BE OF THE SAME SIZE */
- /* To get the choice of the operation from the user */

/* Problem: This programs fills an array with a value submitted by the user. /* array parameter can be expressed as a pointer */

ar-4

```
#include <stdio.h>
/* array parameter */
/* can be expressed as a pointer */
void
fill_array (int *list, int n, int in_value)
{
        int i;
        for (i=0; i<n; ++i)
                list[i] = in_value;
}
int
main (void)
{       int x[10];
        int i;

        /* &x[0] is the address of the x[0] */
        /* which is the same thing as x */
    fill_array (&x[0], 10, 5);


        /* printing the array for verification */
        for (i=0; i<10; ++i)
                printf ("%d  ", x[i]);

        return (0);}
```

/* THE BASIC SEARCH ALGORITHM FOR A 1-D ARRAY -ar-5*/

```
#include <stdio.h>
 int search (const int *arraytosearch, int valuetosearch, int size)
{           int i, found;
            /* initialize found at -1, if value not found, stays at -1 */
            found = -1;
            /* search until found or until end of array */
            i = 0;
            while (found<0 && i<size)
            {
                        if (arraytosearch[i] == valuetosearch)
                                    found = i; /* I have found it! */
                        else
                                    i = i + 1;        }
            return (found);}
int main (void)
{
            int x[] = {12,67,56,60,88,34,123};
            int value = 12;
            int pos, i;
            pos = search (x, value, 7);
            if (pos >= 0)
                        printf ("%d was found at position %d.\n", value, pos);
            else
                        printf ("%d was not found in the array.\n", value);
            return (0);
}
```

Q? HOMEWORK!!!

- Write a program which initializes parts of two arrays of integer grades at the time of their declaration, each with 6 values. Then it calls a function to find, return, and print the highest grade in each of the two arrays.

uses typecasting in the function call and a pointer with offsets inside the function

```
#include <stdio.h>
#define NUM_STUDENTS1 25
#define NUM_TESTS1 10
#define NUM_STUDENTS2 15
#define NUM_TESTS2 8
int get_highest(int *a, int cols, int row, int col);
int main() {    int grades1[NUM_STUDENTS1][NUM_TESTS1] = { {55, 60, 65},  {85, 90, 95} };
  int num_students1 = 2;
  int num_tests1 = 3;      int high_test1;
  int grades2[NUM_STUDENTS2][NUM_TESTS2] = { {55, 60}, {85, 90}, {75, 70} };
  int num_students2 = 3;
  int num_tests2 = 2;
  int high_test2;
    high_test1 =     get_highest( (int *)grades1, NUM_TESTS1, num_students1, num_tests1);
  printf("The highest score in the first class is %d.\n", high_test1);
  high_test2 =       get_highest( (int *)grades2, NUM_TESTS2, num_students2, num_tests2);
  printf("The highest score in the second class is %d.\n", high_test2);
  return 0;}
int get_highest(int *a, int cols, int row, int col)
/* Assumes that the elements with usable values are    in the upper left corner of the array as   delimited by row and col;
  cols is the number of columns in the array passed */
{    int i, j;         int highest = *a;
  for( i = 0; i < row; i++)
    for( j = 0; j < col; j++)
      if ( *(a + i*cols + j) > highest)
        highest = *(a + i*cols + j);
  return highest;}
```

/* PROGRAM # 58 */

```
/* REMINDER FROM CHAPTER 4*/
/* READING DATA FROM A DATA FILE */
/* USE OF EOF  – END OF FILE */
#include<stdio.h>
main( ){
        int x, y, sum, count;
        while( scanf("%d", &x) != EOF ){
                printf("x is %d\n", x);
                sum=0;
                for (count=1; count<=x; count++){
                        scanf("%d", &y);
                        sum=sum+y;
                }
                printf("sum is %d\n", sum);
        }
}
```

ASSUME THAT DATA2 ISYOUR DATA FILE.

data2

1 5

2 5 6

3 5 6 7

4 5 6 7 8

Q???

- Write a complete program reading from a file numbers (the first number being the quantity of the other numbers to read).
- Place these last numbers in an array. All numbers are integers. Write a function taking this array as input and returning the largest number as output.
- The function prototype will be **int largest (int array[ ], int size);**

```
#include <stdio.h>
int  largest (int array[ ], int size) {
int i;
int larget;
largest = array[0];
for (i=0; i<size; +i)
if (array[i]>large)
large=array[i];
return (large);
}
int  main (void) {
int n, array[100], i, big;
FILE *in;
in = fopen ("file.txt", "r");
fscanf (in, "%d", &n;);
for (i=0; i<n; +i)
fscanf (in, "%d", &array[i]);
big = largest (array, n);
printf ("The largest number is %d", big);
fclose (in);
return (0);
}
```

Q?

- **The following program demonstrates multiplication of a square matrix and a scalar array, and 2 square matricies*/**

```
#include <stdio.h>
#define DIM 3
/* matrix by a scalar */
void m1 ( ) {
int M[DIM][DIM] = {{3,1,9}, {8,4,5}, {6,2,7}};
int N[DIM][1] = {4,3,5};
int prod[DIM];
int i, k, j;        j = 0;
printf ("\n - - - Matrix * Scalar - - -\n");          /* perform the multiplication */
for (i=0; i<DIM; ++i) {
prod[i] = 0;
for (k=0; k<DIM; ++k) {
prod[i] = prod[i] + M[i][k] * N[j][k]; } }          /* display scalar */
printf ("The Scalar is:\n");
for (k=0; k<DIM; ++k) {
printf ("%d\n", N[j][k]); }
printf ("\n");  /* display matrix 2 */
printf ("The matrix 2 is:\n");
for (i=0; i<DIM; ++i) {
for (k=0; k<DIM; ++k)
printf ("%d\t", M[i][k]);          printf ("\n"); }
printf ("\n");  /* display product matrix */
printf ("The product is:\n");
for (i=0; i<DIM; ++i) {  printf ("%d\t", prod[i]);}
printf ("\n"); }
```

```
/* matrix by a matrix */
void m2 () {
int M[DIM][DIM] = {{3,1,9}, {8,4,5}, {6,2,7}};
int N[DIM][DIM] = {{4,9,2}, {3,7,1}, {5,6,8}};
int prod2[DIM][DIM];
int i, k, j; j = 0;     printf ("\n - - - Matrix * Matrix - - -\n");
/* perform the multiplication */
for (i=0; i<DIM; ++i) {
for (k=0; k<DIM; ++k) {
prod2[i][k] = 0;
for (j=0; j<DIM; ++j) {
prod2[i][k] = prod2[i][k] + M[i][j] * N[j][k]; } } } /* display matrix 1 */
printf ("The matrix 1 is:\n");
for (i=0; i<DIM; ++i) {
for (k=0; k<DIM; ++k)
printf ("%d\t", M[i][k]);        printf ("\n"); }
printf ("\n"); /* display matrix 2 */
printf ("The matrix 2 is:\n");
for (i=0; i<DIM; ++i) {
for (k=0; k<DIM; ++k)
printf ("%d\t", N[i][k]);  printf ("\n"); } printf ("\n"); /* display matrix */
printf ("The product is:\n");  for (i=0; i<DIM; ++i)  {  for (k=0; k<DIM; ++k)
printf ("%d\t", prod2[i][k]);  printf ("\n"); } printf ("\n"); } int main (void){
m1();
m2();
return (0); }
```

/* PROGRAM */

- Write a program that asks for three int numbers, A, B and C. The program then will ask the user to input A int numbers (eg, if A is 5 then user needs to input 5 integer numbers. If these int numbers fall between the interval defined by B and C, then their average is calculated and the result will be returned to the main.

```
#include<stdio.h>
int
main(void ){
        int A, B, C, i, anum, sum=0, count=0;
        float average;
      /* Input number of integers user should enter and ask the interval */
        printf("How many numbers should the user input? ");
        scanf("%d", &A);
        printf("Pls define the interval: ");
        scanf("%d%d", &B, &C);


 for(i=1; i<=A; i++){
            printf("Pls enter an int number; ");
            scanf("%d", &anum);

            if ( anum>=B && anum<=C){
                sum+=anum;
               count++;
}}
                    average=(float) sum/count;

        printf("The average is %.2f.\n", average);

        return(0);

}
```

- /* sums and averages */ /* this program calculates the average of all the numbers in a file of integers */

/* sums and averages */ /* this program calculates the average of all the numbers in a file of integers */

```
#include <stdio.h>
int
main (void){
        int number, sum, count;
        double average;
        FILE *in;
        in = fopen ("numbers.data.txt", "r");
        /* sums and counts must always be initialized to zero */
        sum = 0;
        count = 0;
        /* read number by number until the end of file */
        while (fscanf (in, "%d", &number) != EOF) {
                /* count and sum the numbers */
                sum = sum + number;
                count = count + 1;          }
        fclose (in);
        /* final report */
        printf ("There are %d numbers in the file.\n", count);
        printf ("The sum of all the numbers is %d.\n", sum);
        average = (double)sum / count;
        printf ("There average of all the numbers is %5.2lf.\n", average);
        return (0);
}
```

Q?

- /* READING AND PRINTING MATRICES USE OF POINTER TO ACCESS MATRIX */

Pointers and 2D Arrays

```
/* PROGRAM # 82  READING AND PRINTING MATRICES USE OF POINTER TO ACCESS MATRIX  */
#include <stdio.h>
#define row 3

#define column 3
void GetMatrix(int *Mat);
void PrintMatrix(int *Mat);
main( ){
        int MatA[row][column]; /* Declare a 2D matrix */
        int *VecA;
                VecA=&MatA[0][0]; /* Get the Pointer to point to the TOP of the Matrix */
        GetMatrix(VecA); /* Call the function to input the matrix from the user */
        PrintMatrix(VecA); /* Call the function to display the matrix */
}       /* To get elements of a matrix from the user */
void GetMatrix(int *Mat){
int i, j;
for(i= 0; i<row; i++)
                for(j= 0; j<column; j++){
                        printf("Matrix[%d][%d]: ", i, j);
        scanf("%d", Mat+(i*column)+j ); /* Refer TO FIG. 1 to verify this statement */
                } }   /* To print a matrix */
void PrintMatrix(int *Mat){
 int i, j;
        for(i= 0; i<row; i++){
                for(j= 0; j<column; j++)
                        printf("%5d", *(Mat+(i*column)+j) );
```

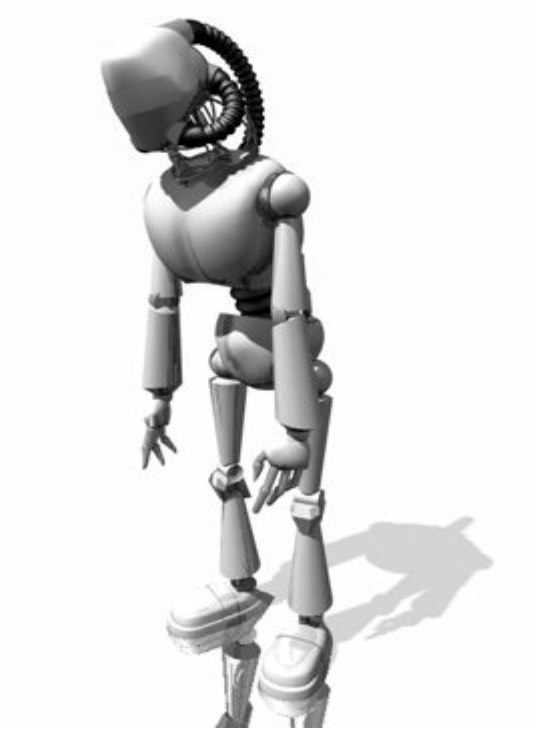
/* READING AND PRINTING MATRICES */
/* USE OF POINTER TO ACCESS MATRIX */

```
/* PROGRAM #82-B */
#include <stdio.h>
#define row 3
#define column 3
void GetMatrix(int *First, int *Last);
void PrintMatrix(int *First, int *Last);
main( ){
        int MatA[row][column];
/* Call functions to get elements of the matrix and display it */
        GetMatrix(&MatA[0][0], &MatA[row-1][column-1]);
        PrintMatrix(&MatA[0][0], &MatA[row-1][column-1]);
} /*To get elements of a matrix from the user */
void GetMatrix(int *First, int *Last){
int *ptr;
for(ptr= First; ptr<= Last; ptr++){
                printf("Enter the Matrix: ");
                scanf("%d", ptr);
        }
}  /*To print a matrix */
void PrintMatrix(int *First, int *Last){
int *ptr;
        for(ptr= First; ptr<= Last; ptr++){
                printf("%5d", *ptr );
```

PROGRAM!!!

/* READING AND PRINTING MATRICES USE OF POINTER TO ACCESS MATRIX */

Using a Function to Read 2 Matrices with Different Dimensions

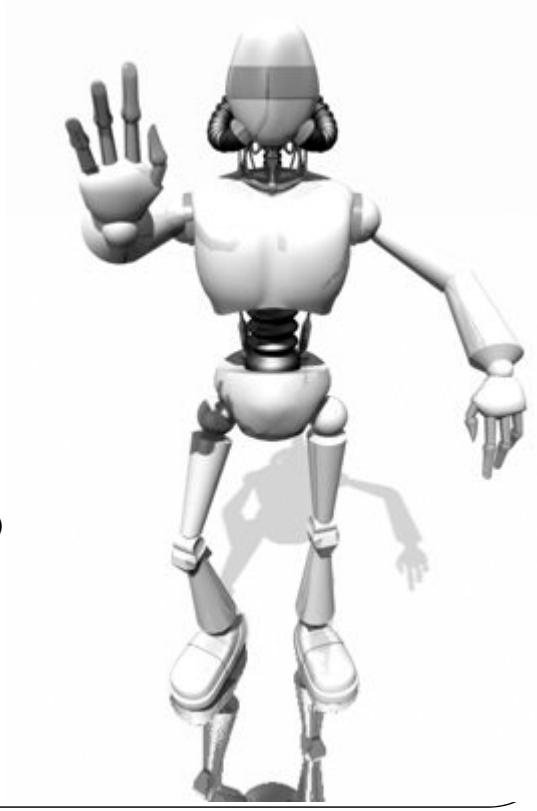


Using a Function to Read 2 Matrices with Different Dimensions

```
/* PROGRAM # 83 */
/* READING AND PRINTING MATRICES USE OF POINTER TO ACCESS MATRIX  */
#include <stdio.h>
#define row1                    3
#define column1         2
#define row2                    2
#define column2         2
void GetMatrix(int *Mat, int r, int c);
void PrintMatrix(int *Mat, int r, int c);
main( ){    /* Declare 2 matrices with different dimensions */
        int MatA[row1][column1], MatB[row2][column2];
        int *VecA, *VecB;
        int DimA1, DimA2, DimB1, DimB2;
        DimA1=row1;
DimA2=column1;
DimB1=row2;
DimB2=column2;
        VecA=&MatA[0][0];  /* Get the Pointer to point to the TOP of the Matrix */
VecB=&MatB[0][0];
GetMatrix(VecA, DimA1, DimA2); /* Input the first matrix with 3 rows and 2 columns*/
PrintMatrix(VecA, DimA1, DimA2);
GetMatrix(VecB, DimB1, DimB2); /* Input the second matrix with 2 rows and 2 columns
*/PrintMatrix(VecB, DimB1, DimB2); }
```

/* PROGRAM # 83 */

```
/*To get elements of a matrix from the user */
void GetMatrix(int *Mat, int r, int c){
int i, j;
for(i= 0; i<r; i++)
                    for(j= 0; j<c; j++){
                              printf("Matix[%d][%d]: ", i, j);
                              scanf("%d", Mat+(i*c)+j );
                    }
}
/*To print a matrix */
void PrintMatrix(int *Mat, int r, int c){
int i, j;
          for(i= 0; i<r; i++){
                    for(j= 0; j<c; j++)
                              printf("%5d", *(Mat+(i*c)+j) )
                    printf("\n");
          }
}
```



/* ADDING 2 MATRICES */

```
/* PROGRAM # 84 */
#include <stdio.h>
#define        row      3
#define        column   5
/*When using a 2D array as an argument,                    */
/*We don't need to indicate the 1st dimension      */
void GetMatrix(int Mat[ ][column]);
void AddMatrices(int A[ ][column], int B[ ][column], int C[ ][column]);
void PrintMatrix(int Mat[ ][column]);
main( ){
        /* Declare three 2D matrices */
        int MatA[row][column], MatB[row][column], MatC[row][column];
        /* Call functions to input two matrices from the user*/
                GetMatrix(MatA);
                GetMatrix(MatB);
        /* Call the function to add the matrices */
                AddMatrices(MatA, MatB, MatC);
        /* Display matrices */
                PrintMatrix(MatA);
                PrintMatrix(MatB);
                PrintMatrix(MatC);
```

/* To get elements of a matrix from the user */

```
/* PROGRAM # 84 */
void GetMatrix(int Mat[ ][column]){

int i, j;
puts("");
for(i= 0; i<row; i++)
                    for(j= 0; j<column; j++){
                              printf("Matrix[%d][%d]: ", i, j);
                              scanf("%d", &Mat[i][j]);
                    } } /*To perform matrix addition */
void AddMatrices(int A[ ][column], int B[ ][column], int C[ ][column]){
          int i, j;
          for(i= 0; i<row; i++)
                    for(j= 0; j<column; j++)
                              C[i][j]=A[i][j]+B[i][j];
} /*To print a matrix */
void PrintMatrix(int Mat[ ][column]){
int i, j;
          for(i= 0; i<row; i++){
                    for(j= 0; j<column; j++)
                              printf("%5d",Mat[i][j]);
                    printf("\n");
          }
```

/* TO MULTIPLY TWO MATRICES */

```
/* PROGRAM # 85 */
/* Mutiply A by B and put the result in C */
/* A is mxn, B is nxp, then C is mxp */
void Multiply(int A[rowA][columnA], int B[rowB ][columnB], int
C[rowC ][columnC]){
        int i,j,k;
        for(i = 0; i < rowA; i++)
                for(j = 0; j < columnB; j++) {
                        C[i][j] = 0;
                        for(k = 0; k < columnA; k++)
                                C[i][j] += A[i][k] * B[k][j];
                }
}
```